

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

“Д35: Технический дизайн микросервисной архитектуры”

Выполнил:

Преображенский Артемий

Группа:

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

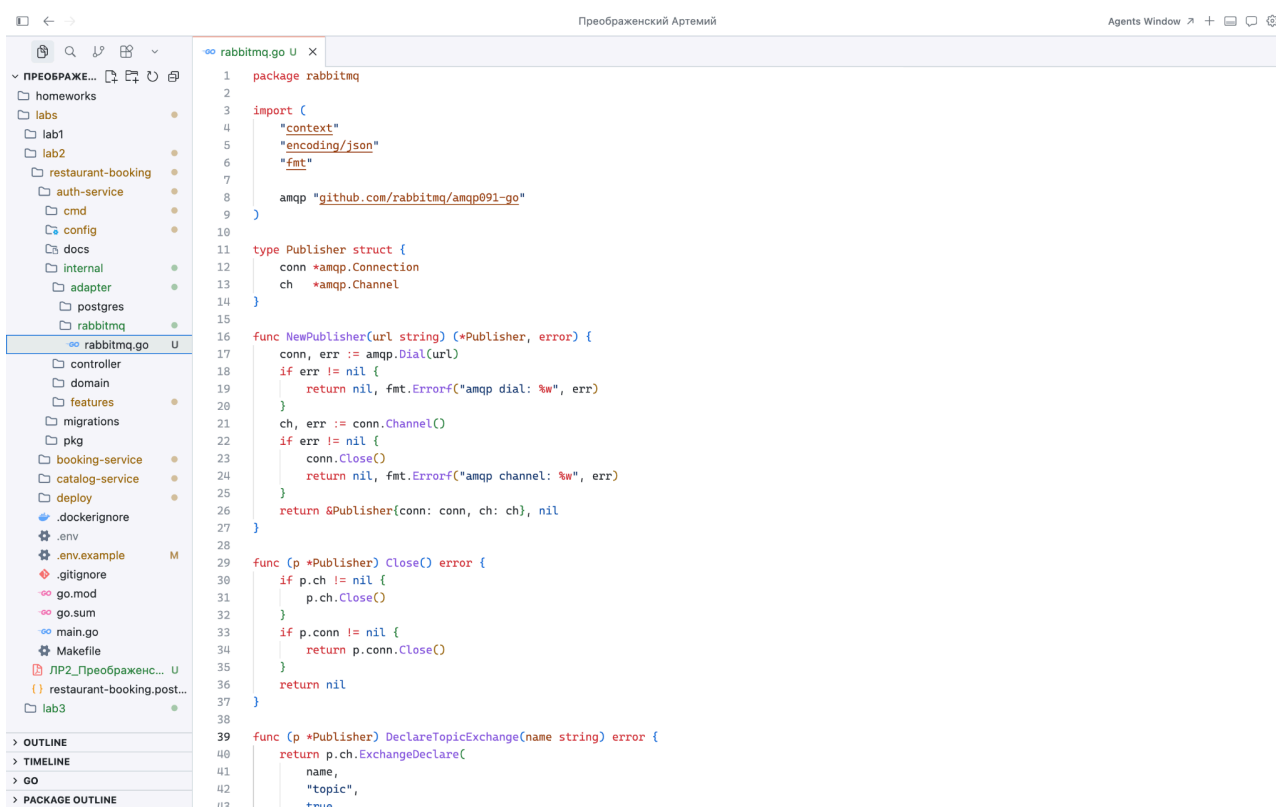
2026 г.

Задача

1. Реализация межсервисного взаимодействия посредством очередей сообщений

Ход работы

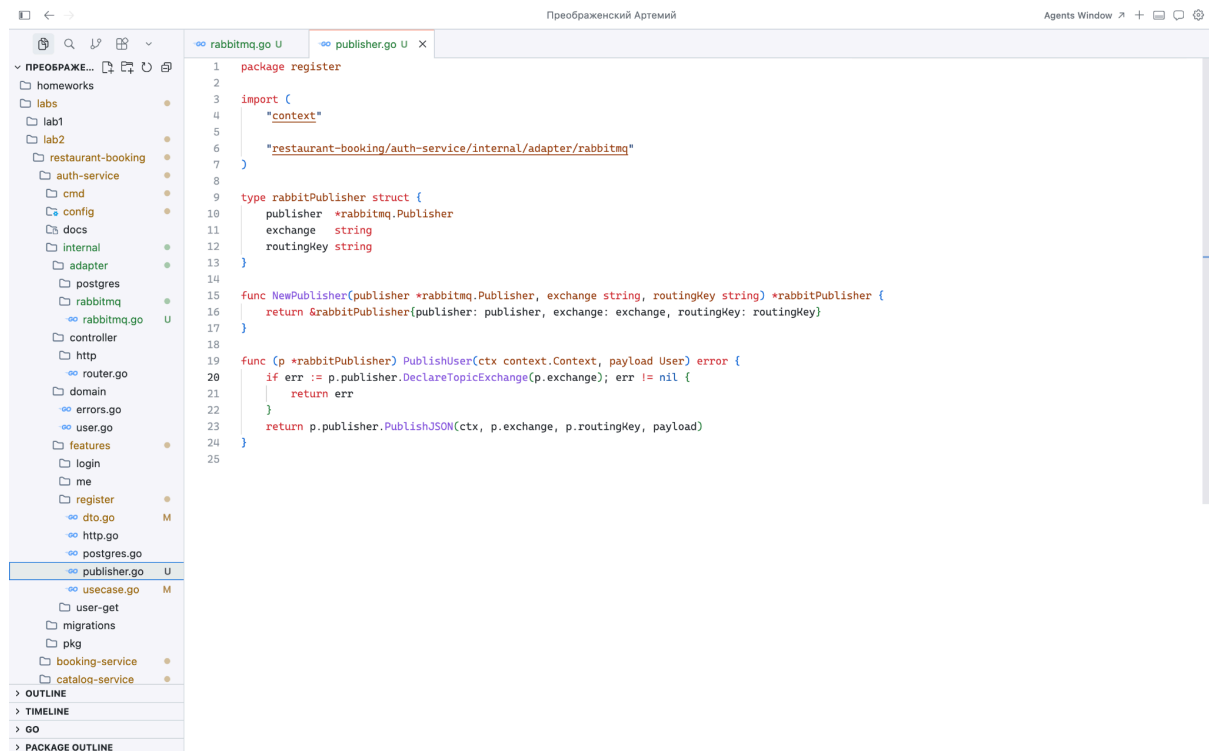
В `deploy/docker-compose.yml` был добавлен контейнер RabbitMQ с настроенными портами и healthcheck. Для `auth-service` и `catalog-service` была указана переменная `RABBITMQ_URL`, а также зависимость от готовности брокера, чтобы сервисы запускались только после того, как RabbitMQ станет доступен. Для обмена сообщениями был определён контракт события: используется topic exchange `restaurant.user` и routing key `user.registered`. В сообщение передаются данные зарегистрированного пользователя в формате JSON, в том числе как минимум `id` и `full_name`. После успешного сохранения пользователя в базе данных `auth-service` публикует устойчивое сообщение в этот exchange.



```
1 package rabbitmq
2
3 import (
4     "context"
5     "encoding/json"
6     "fmt"
7
8     amqp "github.com/rabbitmq/amqp091-go"
9 )
10
11 type Publisher struct {
12     conn *amqp.Connection
13     ch   *amqp.Channel
14 }
15
16 func NewPublisher(url string) (*Publisher, error) {
17     conn, err := amqp.Dial(url)
18     if err != nil {
19         return nil, fmt.Errorf("amqp dial: %w", err)
20     }
21     ch, err := conn.Channel()
22     if err != nil {
23         conn.Close()
24         return nil, fmt.Errorf("amqp channel: %w", err)
25     }
26     return &Publisher{conn: conn, ch: ch}, nil
27 }
28
29 func (p *Publisher) Close() error {
30     if p.ch != nil {
31         p.ch.Close()
32     }
33     if p.conn != nil {
34         p.conn.Close()
35     }
36     return nil
37 }
38
39 func (p *Publisher) DeclareTopicExchange(name string) error {
40     return p.ch.ExchangeDeclare(
41         name,
42         "topic",
43         true,
```

Со стороны `catalog-service` при запуске поднимается consumer. Он объявляет ту же топологию RabbitMQ, привязывает очередь к exchange и в отдельной горутине начинает чтение сообщений. После получения

сообщения выполняется разбор JSON в структуру domain.User, затем вызывается UpsertUser, где данные записываются или обновляются в таблице users_cache базы catalog_db.



```
1 package register
2
3 import (
4     "context"
5
6     "restaurant-booking/auth-service/internal/adapter/rabbitmq"
7 )
8
9 type rabbitPublisher struct {
10     publisher *rabbitmq.Publisher
11     exchange string
12     routingKey string
13 }
14
15 func NewPublisher(publisher *rabbitmq.Publisher, exchange string, routingKey string) *rabbitPublisher {
16     return &rabbitPublisher{publisher: publisher, exchange: exchange, routingKey: routingKey}
17 }
18
19 func (p *rabbitPublisher) PublishUser(ctx context.Context, payload User) error {
20     if err := p.publisher.DeclareTopicExchange(p.exchange); err != nil {
21         return err
22     }
23     return p.publisher.PublishJSON(ctx, p.exchange, p.routingKey, payload)
24 }
25
```

Вывод

В дз5 подключён RabbitMQ, настроены exchange, очередь и binding, реализованы адаптеры AMQP в auth (публикация при регистрации) и catalog (подписка и обновление users_cache).